



Lisk Summer Bootcamp

The facilitator

Bratipah Kioko (Alienate)

Web3 Developer and DevRel

Twitter: @Alienate_B

LinkedIn: Bratipah Kioko



Today's Agenda

1. Introduction to dapps
2. Web3 Libraries: Introduction to Ether.js
3. Connecting to the Blockchain
4. Interacting with Smart Contracts
5. Reading data from the smart contract
6. Displaying Blockchain data
7. Scaffold Lisk



Introduction to dapps

Definition of dapps

dApps are blockchain-based applications that operate without a central authority.
Common use cases include:

- **Decentralized Finance (DeFi)** – Lending, borrowing, and trading without banks (e.g., Uniswap, Aave, Oku.trade, Velodrome).
- **Real World Assets** - Tokenization of assets (e.g. Backed, Nomachain)
- **NFT Marketplaces** – Buying, selling, and minting unique digital assets (e.g., OpenSea, Blur, Rarible).
- **Gaming** – Play-to-earn games and virtual economies (e.g., Axie Infinity, Decentraland, Pokecoin).
- **Supply Chain Management** – Transparent and trackable logistics (e.g., VeChain, Afrikabaal).

Key Characteristics of Dapps

1. Decentralization

- DApps run on a peer-to-peer (P2P) blockchain network rather than centralized servers. No single entity has full control, reducing censorship and single points of failure.

2. Open Source

- Most DApps are open-source, allowing anyone to inspect, modify, and contribute to the code. Transparency ensures trust among users.

3. Interoperability

- Many DApps are designed to work with other blockchain protocols (e.g., via cross-chain bridges).

4. Cryptographic Security

- Data is secured via cryptographic hashing and consensus mechanisms (e.g., Proof of Work, Proof of Stake). Users control their private keys, enhancing security.

5. Tokenized Incentives

- Many Dapps have native tokens (e.g., ETH for Ethereum-based DApps, LSK for Lisk Dapps) to incentivize participation. Users may earn tokens via staking, liquidity mining, or other mechanisms.

Web3 Libraries

Web3 Libraries

Web3 libraries are essential tools for interacting with blockchain networks like Lisk. They allow developers to connect to nodes, send transactions, deploy smart contracts, and read on-chain data. Below is an overview of the most popular Web3 libraries, their features, and use cases.

Examples

- [Ether.js](#) - a JavaScript library that allows developers to interact with EVM-compatible blockchain networks.
- [Web3.js](#) - a javascript library that interacts with evm networks
- Viem - a TypeScript interface for Ethereum that provides low-level stateless primitives for interacting with Ethereum.

[Read more](#)

Comparison Table

Library	Language	Size	Best For	Key Strengths
Ethers.js	Js/Ts	~150 KB	Modern apps(metamask, Tally)	Lightweight, clean API, built-in wallet class
Web3.js	Js	~ 1 MB	Full Node access	Legacy, feature-rich, relies on external providers
Viewm	Ts	~30 KB	High Performance	Fast, Wagmi compatible

Ethers.Js

Core Components

- Providers connect to ethereum nodes (e.g via Json-rpc, Infura, Alchemy)
- Signers represents Ethereum accounts and can sign transactions / messages
- Contracts interacts with smart contracts using ABI & an Ethereum address
- Use cases of [ethers.js](#) include sending ETH/LSK, Smart contract Interactions & signing and verifying messages

[Read more](#)

Connecting to the Blockchain

- .Connecting to the Lisk Network using your preferred Wallet

Network Name - Lisk Sepolia Testnet

RPC Endpoint - <https://rpc.sepolia-api.lisk.com>

Chain ID - 4202

Currency Symbol - ETH

Block Explorer - <https://sepolia-blockscout.lisk.com>

- Add Lsk & Eth to the Lisk network based wallet
 - <https://docs.lisk.com/network-info>
- Receive LSK testnet tokens from faucets
 - <https://console.optimism.io/faucet>
 - <https://sepolia-faucet.lisk.com/>

[Read More](#)

Interacting with the Smart Contract

Interacting with the Smart Contract

- Deploy Smart Contract
 - a. `npx hardhat run scripts/deploy.ts --network lisk-sepolia`
- Verify Smart Contract
 - a. `npx hardhat verify --network lisk-sepolia <deployed address>`
- Uses the Blockscout Lisk to be able to read & write contracts information
 - a. <https://sepolia-blockscout.lisk.com/>
- Install [ethers.js](#) in a project you can use a CDN link or npm install --save ethers
 - a.

```
<script src="https://cdn.ethers.io/lib/ethers-5.2.umd.min.js" type="application/javascript" ></script>
```

[Read more](#)

Reading data from the Smart Contract

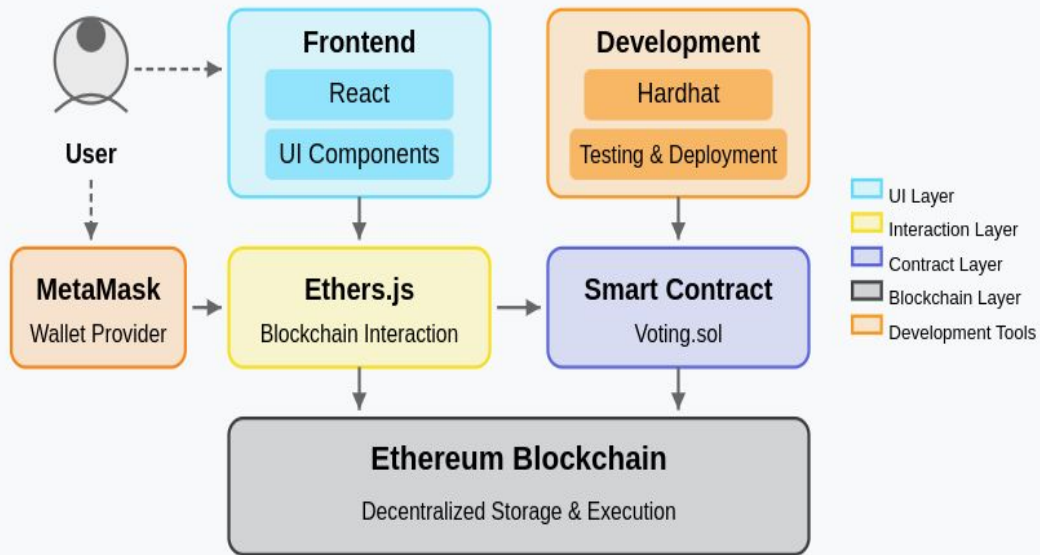
Reading data from Smart Contracts

Reading data from smart contracts with ethers.js is straightforward once you have the contract address and ABI. The library handles all the low-level details, allowing you to focus on your application logic.

Remember that read operations don't require gas or a signer since they don't modify the blockchain state.

Displaying Blockchain data

Blockchain Application Architecture



Scaffold Lisk

Scaffold Lisk

Scaffold Lisk is a fork of Scaffold-OP with minimal differences, providing additional dApp examples, native support for Superchain testnets, and more low-level instructions. We highly recommend the Scaffold-ETH2 docs as the primary guideline.

- Built using NextJS, RainbowKit, Hardhat, Wagmi, Viem, and Typescript.

```
git clone https://github.com/LiskHQ/scaffold-lisk.git
```

```
cd scaffold-lisk
```

```
yarn install
```

```
Yarn chain
```

```
Yarn deploy
```

Hands-on Demonstration

Summary

1. Introduction to dapps
2. Web3 Libraries: Introduction to Ether.js
3. Connecting to the Blockchain
4. Interacting with Smart Contracts
5. Reading data from the smart contract
6. Displaying Blockchain data
7. Scaffold Lisk



The End

Questions?